Destructuring

Unpack, merge, copy, and the shallow copy trap

1 Destructuring

```
const { a, b = 2 } = { a: 1 };
// a = 1, b = 2 (default)
const { n = 5 } = { n: null };
// n = null (no default)
const [x, y] = new Set([1, 2]);
// x = 1, y = 2 (any iterable)
const { u: id } = { u: 7 };
// id = 7 (rename)
```

2 Spread & rest

```
const a = [1], o = { p: 1 };
[...a, 2]; // [1, 2]
({ ...o, p: 9 }); // { p: 9 }
const [h, ...rest] = [7, 8, 9];
// h = 7, rest = [8, 9]
({ ...null }); // {}
[...null]; // TypeError
```

	Shallow	Deep
Syntax	`{ ...o }`	`structuredClone(o)`
Top level	✓ copied	✓ copied
Nested	✗ shared	✓ copied

Also shallow: **Object.assign**, **[...a]**, **slice()**. **structuredClone** throws on functions.

TRAP What does this log?

```
const a = { n: { v: 1 } };
const b = { ...a };
b.n.v = 2;
console.log(a.n.v);
```

2

Spread copies one level. **n** is the same object in both.

KEY TAKEAWAY

- `{ ...o }` Copies one level only
- `deep` structuredClone(o)
- `...rest` Collects the rest, last

? Follow-up questions to expect

- **Destructuring `null` or `undefined`?** → TypeError. For parameters write f({ x } = {}).
- **`{ ...null }` vs `[...null]`?** → Object spread gives {}, array spread throws TypeError.
- **Does spread keep the prototype?** → No. Only own enumerable properties are copied.
- **How do you read nested data?** → Mirror the shape: { user: { name } }.